

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

7. Ingesta de datos II	1
7.1. Apache Pig	2
7.1.1. Arquitectura	3
7.1.2. Modos de Ejecución	4
7.1.3. Pig Latin	4
7.2. Apache Flume	9
7.2.1. Arquitectura	9
7.2.2. Fiabilidad	12
7.2.3. Particiones e Interceptores	12
7.2.4. Configuraciones avanzadas	13

Listado de acrónimos

ETL	Extraction Transformation and Load
HDFS	Hadoop Distributed File System
JDBC	Java Database Connector
JVM	Java Virtual Machine
RDBMS	Relational Database Management System
UDF	User Defined Function

7

Capítulo

Ingesta de datos II

Francisco P. Romero

MapReduce es un framework de programación basado en el principio de Divide y vencerás en la que se permite al programador desarrollar soluciones a base de especificar una función *map* de división de los datos y una función *reduce* de combinación, reducción y obtención de resultados. Es un marco de trabajo eficiente y especialmente orientado a la aplicación del paralelismo para mejorar la eficiencia en la resolución de problemas, pero no siempre es obvio encajar este tipo de procesamiento, sobre todo en cuanto a datos, en este patrón, que puede dar lugar a diferentes operaciones de mapeo y reducción que dificultan su aplicación.

Una de las abstracciones más destacadas de MapReduce es Pig[ORS⁺08]. Esta herramienta permite al el desarrollador especificar secuencias de transformaciones en un lenguaje procedimental de alto nivel llamado Pig Latin. Pig está especialmente orientado al procesamiento de datos en paralelo y puede operar con datos relacionales, anidados y no estructurados. Dispone de funciones integradas para unir, agrupar, ordenar, filtrado, etc. y también puede ampliarse mediante funciones definidas por el usuario (UDF). Pig es una de las herramientas más populares para ejecutar los procesos de transformación y limpieza de datos en los flujos ETL.

Por otro lado, Apache Flume es un producto que forma parte del ecosistema Hadoop, y conforma una solución Java distribuida y de alta disponibilidad para recolectar, agregar y mover grandes cantidades de datos desde diferentes fuentes a un almacenamiento de datos.

Mediante el uso de estas dos aplicaciones, añadidas al resto del ecosistema Hadoop se va conformando un flujo de trabajo con el fin de ofrecer soluciones Big Data integrales, es decir, que cubran la funcionalidad desde que se captura la información (Flume), hasta que se almacena y consulta (Hive, Impala), posibilitando un procesamiento rápido y sencillo de programar (Pig)

7.1. Apache Pig

Apache Pig ofrece un nivel de abstracción que MapReduce para el procesamiento de grandes conjuntos de datos. Con Pig, se pueden utilizar estructuras de datos mucho más complejas, ya que suelen ser multivaluadas y anidadas, y las transformaciones que se pueden aplicar a los datos son mucho más útiles, como por ejemplo las uniones o joins que son no son triviales de implementar en MapReduce [Vad16].

Pig se compone de dos partes:

- El lenguaje utilizado para expresar los flujos de datos, llamado **Pig Latin**.
- El entorno de ejecución para ejecutar los programas Pig Latin. Actualmente existen dos entornos: la ejecución local en una única JVM y la ejecución distribuida en un clúster de Hadoop.

Un programa *PigLatin* se compone de una serie de operaciones, o transformaciones, que aplicadas a los datos de entrada para producir la salida. En su conjunto, las operaciones describen un flujo de datos, que el entorno de ejecución de Pig traduce en una representación ejecutable y lo ejecuta. Pig se encarga de convertir las transformaciones en una serie de trabajos de MapReduce, lo que le permite focalizarse en los datos más que en la paralelización de las tareas.

Las características principales de Apache Pig son las siguientes:

- *Fácil de programar*: Para los no programadores la elaboración de soluciones en Java dentro del paradigma MapReduce es una tarea demasiado ardua. Pig facilita el proceso de programación mediante la conversión de las consultas y los procedimientos que se ejecutan sobre los datos en procesos MapReduce de forma interna y transparente al usuario.
- *Operaciones basadas en datos*: Contiene un conjunto de operaciones disponibles especialmente adecuados para el tratamiento de datos como ordenaciones, filtrados y uniones.
- *Optimización*: La organización del código y de las operaciones permite al sistema optimizar la ejecución de los diferentes procedimientos de forma automática, permitiendo al usuario centrarse en la semántica de las operaciones y no en su eficiencia.
- *Extensibilidad*: Permite la creación de funciones definidas por el usuario (UDF) en las que se pueden incluir procedimientos propios a ejecutar sobre los datos.
- *Flexibilidad*: Puede manejar de forma sencilla tanto datos estructurados como datos no estructurados.

En resumen, Pig ofrece la posibilidad de especificar en un menor número de líneas de código operaciones complejas sobre los datos mediante una serie de operaciones especialmente diseñadas para el procesamiento de datos.

7.1.1. Arquitectura

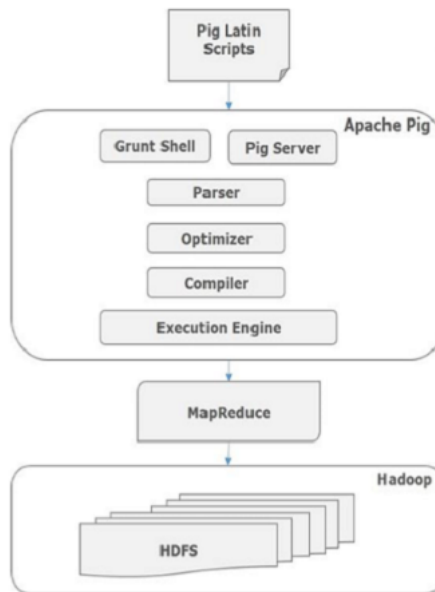


Figura 7.1: Arquitectura Apache Pig. Fuente: Apache

Dentro del framework Pig se pueden distinguir diferentes componentes:

- **Parser:** Inicialmente los *Pig Scripts* son manejados por el parseador. Éste verifica la sintaxis del script, etc. La salida del parseador será un Grafo Dirigido Acíclico (DAG) que representa la ejecución de las sentencias escritas en Pig Latin y las operaciones lógicas. En este grafo, las operaciones lógicas se representan como nodos y los flujos de datos como aristas.
- **Optimizer:** El plan lógico (DAG) es posteriormente procesado por un optimizador. Para ello, Pig aplica reglas de optimización para dividir, fusionar, transformar y reordenar los operadores. De esta forma, un plan lógico se convierte en un plan MapReduce, que es una vista fragmentada de la consulta de Pig con los límites del trabajo MapReduce definidos. A partir de ese momento se realizan nuevas optimizaciones. La mayoría de las reglas de optimización de la capa lógica tienen como objetivo reducir la cantidad de datos en la fase de mezcla o los archivos temporales generados entre trabajos MapReduce consecutivos.
- **Compilador:** El compilador compila el plan optimizado realizado anteriormente y lo traduce directamente a una serie de trabajos MapReduce.
- **Motor de Ejecución:** Después de realizar todas las operaciones anteriores, estos trabajos Map Reduce se envían al motor de ejecución, que se ejecuta en la plataforma Hadoop para producir los resultados deseados.

7.1.2. Modos de Ejecución

- **Modo Local:** En este modo, Pig se ejecuta en una única máquina virtual Java (JVM) y utiliza el sistema de archivos local, es decir, todos los procesos se ejecutan utilizando *localhost* y los datos de entrada y salida están almacenados en el sistema de archivos local. Este modo únicamente se recomienda para el análisis de pequeños conjuntos de datos, pruebas o aprendizaje. Para ejecutar en modo local el comando es el siguiente:

```
$ pig -x local
```

- **Modo Map Reduce:** En este modo, los scripts escritos en PigLatin son traducidos a trabajos MapReduce y ejecutados en un clase Hadoop (en modo pseudo o completamente distribuido). Es el que se debe utilizar para procesar grandes cantidades de datos. El comando para su utilización es el siguiente:

Además, hay diferentes formas de ejecutar un programa Pig en cualquiera de los modos de ejecución anteriores.

- **Modo Interactivo:** En este modo, los comandos y operaciones Pig se ejecutan en el shell *Grunt* que se invoca con el comando *pig*. Una vez que aparece el *prompt* de *grunt* se pueden ejecutar sentencias PigLatin de forma interactiva.
- **Modo por lotes/Batch Mode:** En este modo podemos ejecutar un fichero que contiene todos los comandos de nuestro programa PigLatin (extensión .pig).
- **Modo Embebido:** En este caso se pueden utilizar funciones desarrolladas por el propio programador utilizando lenguajes de alto nivel como Python o Java.

7.1.3. Pig Latin

Un programa Pig Latin consiste en una colección de sentencias (*statement*). Una sentencia puede ser considerada como una operación o un comando. Las declaraciones suelen terminar con un punto y coma. Por otro lado, el Pig Latin tiene dos formas de comentarios. Los guiones dobles `--` se utilizan para los comentarios de una sola línea. El intérprete de Pig Latin ignora todo lo que va desde el primer guión hasta el final de la línea. Los comentarios multi-línea se hacen con `/*` y `*/` y delimitan el principio y el final del bloque de comentarios. Pueden abarcar varias líneas o estar incrustados en una sola línea:

Pig Latin tiene una lista de palabras clave que tienen un significado especial en el lenguaje y no pueden utilizarse como identificadores. Entre ellas se encuentran los operadores (*LOAD*), los comandos (*cat*, *ls*), expresiones (*FLATTEN*), y funciones (*DIFF*, *MAX*). Pig Latin tiene reglas mixtas sobre la sensibilidad a las mayúsculas y minúsculas. Los operadores y los comandos no distinguen (para que el uso interactivo más sencillo); sin embargo, los alias y los nombres de funciones son sensibles a las mayúsculas y minúsculas.

Modelo de datos

El modelo de datos de Pig Latin está completamente anidado y puede tratar tanto tipos de datos atómicos como enteros y flotantes, como tipos de datos complejos no atómicos como mapa (*map*) y tupla (*tuple*). Sus componentes son los siguientes:

- **Átomo:** Un átomo es un único valor almacenado en forma de cadena y puede ser utilizado como un número y una cadena. Los valores atómicos de Pig son integer, double, float, byte array y char array. Un único valor atómico también se denomina campo. Por ejemplo, *Felipe* o 51.
- **Tupla** - Una tupla es un registro que contiene un conjunto ordenado de campos (de cualquier tipo). Una tupla es muy similar a una fila en un RDBMS. Por ejemplo, (*Felipe*, 51)
- **Bolsa (Bag)** - Un átomo es un valor único almacenado en forma de cadena y puede ser utilizado como un número y una cadena. Los valores atómicos de Pig son el entero, el doble, el flotante, el array de bytes y el array de char. Un único valor atómico también se llama campo. Por ejemplo,

$[(Felipe, 51), (Leticia, 45)]$

- **Mapa** - Un conjunto de pares clave-valor se conoce como mapa. La clave debe ser única y debe ser de tipo char array. Sin embargo, el valor puede ser de cualquier tipo. Por ejemplo,

$[nombre : Felipe, edad : 51]$

- **Relación** - Un conjunto de tuplas se denomina relación.

Sentencias

Las sentencias Pig Latin son las construcciones básicas que se utilizan para procesar datos con Pig. Las sentencias Pig Latin pueden abarcar varias líneas y deben terminar con un punto y coma (;). Por defecto, las sentencias Pig Latin se procesan utilizando la ejecución de consultas múltiples.

Una sentencia Pig Latin es un operador que toma una relación como entrada y produce otra relación como salida. Esta definición se aplica a todos los operadores de Pig Latin excepto LOAD y STORE que leen datos desde y escriben datos en el sistema de archivos. Las sentencias Pig Latin pueden incluir expresiones y esquemas.



Los esquemas permiten asignar nombres y declarar tipos para los campos. Los esquemas son opcionales, pero es muy recomendable su uso dado que las declaraciones de tipos dan lugar a una mejor comprobación de errores en tiempo de análisis y a una ejecución más eficiente del código.

Los programas Pig Latin se organizan generalmente de la siguiente manera:

- **Extracción:** Una sentencia LOAD para leer datos del sistema de archivos, normalmente apoyadas por diferentes funciones específicas de lectura y almacenamiento de datos. De entre éstas últimas la más popular es *PigStorage*
- **Transformación:** Pig cuenta con amplio conjunto de sentencias para el procesamiento y la transformación de los datos. Las operaciones más usadas son las siguientes:
 - *FILTER*: Selecciona tuplas de una relación basándose en alguna condición. Trabaja con tuplas o con filas de datos.
 - *FOREACH*: Genera transformaciones de datos basadas en columnas de datos. Se utiliza la combinación *FOREACH . . . GENERATE* para trabajar con columnas de datos.
 - *GROUP/COGROUP*: Agrupa los datos en una o varias relaciones. Son operaciones idénticas y pueden trabajar con una o más relaciones. Para facilitar la lectura, GROUP se utiliza en las sentencias que implican una relación y COGROUP se utiliza en las sentencias que implican dos o más relaciones.
 - *JOIN* (inner/outer): Realiza una unión externa de dos relaciones basada en valores de campo comunes.
 - *UNION*: Fusiona el contenido de dos o más relaciones.
 - *SPLIT*: Divide el contenido de una relación en varias relaciones.
- **Carga/Almacenamiento/Diagnóstico:** Se utiliza el operador STORE y las funciones load/store para escribir los resultados en el sistema de archivos (*PigStorage* es la función de almacenamiento por defecto). Durante la fase de prueba/depuración de su implementación se puede utilizar DUMP para mostrar los resultados en consola. Sin embargo, en un entorno de producción siempre se debe utilizar STORE para almacenar los resultados.

A medida que se ejecuta un programa Pig Latin, cada declaración se analiza sucesivamente. Si hay errores de sintaxis u otros problemas (semánticos), como alias indefinidos, el intérprete se detendrá y mostrará un mensaje de error. El intérprete construye un plan lógico para cada operación relacional que forma el núcleo de un programa Pig Latin. Este plan se añade al plan lógico para el programa hasta ahora y entonces el intérprete pasa a la siguiente sentencia. Pig Latin proporciona operadores para ayudar la depuración de los programas, estos son:

- *DUMP* para mostrar por consola los resultados en la pantalla.
- *DESCRIBE* para revisar el esquema de una relación.
- *EXPLAIN* para analizar los planes de ejecución lógicos, físicos o map reduce para calcular una relación.
- *ILLUSTRATE* para ver la ejecución paso a paso de una serie de sentencias.

Por otro lado, también remarcar que Pig almacena los datos intermedios generados entre los trabajos de MapReduce en una ubicación temporal en HDFS. Esta ubicación debe existir ya en HDFS antes de su uso. Esta ubicación puede ser configurada usando la propiedad *pig.temp.dir*. El valor por defecto de la propiedad es */tmp*".

```
A = LOAD 'client' USING PigStorage(',')  
  AS (id:int, nombre:chararray, edad:int);  
B = FOREACH A GENERATE nombre;  
DUMP B;
```

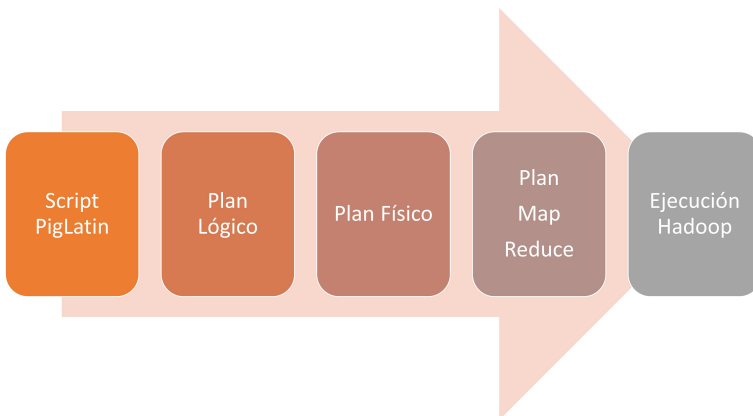


Figura 7.2: Ejecución de Sentencias en Pig

Tabla 7.1: Tabla de sentencias de PigLatin

Categoría	Operación	Descripción
Loading and storing	LOAD	Carga los datos a una relación
	STORE	Almacena una relación
	DUMP	Muestra una relación por consola
Filtrado	FILTER	Elimina registros de una relación de acuerdo a una condición
	DISTINCT	Elimina registros duplicados de una relación
	FOREACH...GENERATE	Añadir o eliminar campos en una relación
	MAPREDUCE	Ejecuta un trabajo MapReduce que toma como entrada una relación
	STREAM	Transforma una relación utilizando un programa externo
	SAMPLE	Extrae una muestra aleatoria de una relación
	ASSERT	Evalua una condición sobre todas las filas de la relación
Agrupación y Unión	JOIN	Unión de una o varias relaciones
	GROUP	Agregación de datos en una relación
	COGROUP	Agregación de datos en dos o más relaciones
	CROSS	Crea el producto cartesiano de una relación
	CUBE	Crea agregaciones para todas las combinaciones de ciertas columnas de la relación
Ordenación	ORDER	Ordena la relación de acuerdo a uno o varios campos
	RANK	Asigna un orden a cada tupla de la relación
	LIMIT	Limita el número máximo de tuplas en la relación
Combinación y División	UNION	Combina dos o más relaciones en una
	SPLIT	Divide una relación en dos o más.

7.2. Apache Flume

Apache Flume [Hof13] es un sistema distribuido, fiable y disponible para recopilar, agregar y trasladar de forma eficiente grandes cantidades de datos de log de muchas fuentes diferentes a un almacén de datos centralizado.

Flume [Flu] está diseñado para la ingestión de grandes volúmenes de datos basados en eventos en Hadoop. Un ejemplo básico de uso de Flume es la recopilación de archivos de log en los servidores web de un banco para luego mover los eventos a nuevos archivos agregados en HDFS para su procesamiento. El destino habitual (o sumidero) es HDFS. Sin embargo, Flume es lo suficientemente flexible suficiente para escribir en otros sistemas, como HBase o Solr.

El uso de Apache Flume no se limita a la agregación de datos de log. Dado que las fuentes de datos son personalizables, Flume puede utilizarse para transportar cantidades masivas de datos de eventos, incluyendo, entre otros, datos de tráfico de red, datos generados por las redes sociales, mensajes de correo electrónico y prácticamente cualquier fuente de datos posible.

7.2.1. Arquitectura

Un agente Flume es un proceso (JVM) que aloja los componentes a través de los cuales fluyen los eventos desde una fuente externa hasta el siguiente destino. Una fuente de Flume consume eventos entregados por una fuente externa como un servidor web. La fuente externa envía eventos a Flume en un formato que es reconocido por la fuente Flume de destino. Cuando una fuente Flume recibe un evento, lo almacena en uno o más canales. El canal es un almacén pasivo que mantiene el evento hasta que es consumido por un sumidero de Flume. El sumidero elimina el evento del canal y lo pone en un repositorio externo como HDFS (a través del sumidero Flume HDFS) o lo reenvía a la fuente Flume del siguiente agente Flume (siguiente salto) en el flujo. La fuente y el sumidero dentro del agente dado se ejecutan de forma asíncrona con los eventos puestos en escena en el canal.

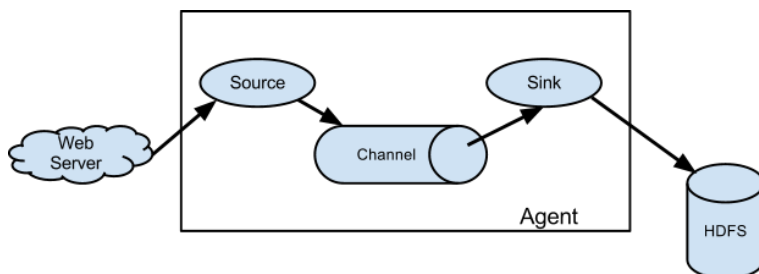


Figura 7.3: Flujo de Trabajo Básico de Flume. Fuente: Apache

Configuración

La configuración controla los tipos de fuentes, sumideros y canales que se utilizan, así como la forma en que se conectan entre sí.

Los nombres de las propiedades forman una jerarquía con el nombre del agente en el nivel superior. En el siguiente ejemplo tenemos un único agente, llamado *agent1*. Los nombres de los diferentes componentes de un agente se definen en el siguiente nivel, por lo que, por ejemplo, *agent1.sources* enumera los nombres de las fuentes que deben ejecutarse en el *agent1* (una única fuente, *source1*). Del mismo modo, el agente tiene un sumidero (*sink1*) y un canal (*channel1*).

```
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1
```

Las propiedades de cada componente se definen en el siguiente nivel de la jerarquía. Las propiedades de configuración de un componente dependen del tipo de componente.

Fuentes (Sources)

Las fuentes (sources) de Flume se encargan de leer datos de clientes externos o de otros sumideros de Flume. La unidad de datos es el *evento*, que es esencialmente una información a utilizar y un conjunto opcional de metadatos. Una fuente de Flume envía estos eventos a uno o más canales de Flume, que se encargan del almacenamiento y del buffering. Flume tiene un amplio conjunto de fuentes incorporadas, incluyendo HTTP, JMS, etc.

Las fuentes de Flume ofrecen las siguientes características :

- Semántica transaccional, que permite que los datos se muevan de forma fiable con una semántica de "al menos una vez".
- Interceptores, que ofrecen la posibilidad de modificar o eliminar eventos. Son útiles para anotar los eventos con el host, la hora y los identificadores únicos, que son útiles para la deduplicación.
- Los selectores, que permiten distribuir o multiplexar los eventos de diversas maneras. En Puede distribuir los eventos replicándolos en varios canales, o puede dirigirlos a diferentes canales basándose en las cabeceras de los eventos.

```
agent1.sources = source1
agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /tmp/spooldir
agent1.sources.source1.channels = channel1
```

Canales (Channels)

Los canales de Flume proporciona capacidad de almacenamiento y persistencia de datos dentro de un agente. Las fuentes publican eventos en un canal y los sumideros consumen eventos de un canal.

Hay tres tipos principales de canales incluidos en Flume:

- *Memoria*: Los canales de memoria almacenan los eventos en una cola en memoria. Esto es muy útil para flujos de datos de alto rendimiento sacrificando la persistencia y la robustez, ya que si un agente se cae se producirá una pérdida de datos.
- *Archivos*. Los canales basados en archivos guardan los eventos en el disco lo cual les permite ofrecer garantías de persistencia y robustez ante fallos.
- Los canales JDBC almacenan los eventos en una base de datos. Esto proporciona grandes capacidades de persistencia, robustez y recuperabilidad, pero a costa de un rendimiento más bajo.

Un ejemplo de canal de memoria se puede observar en la siguiente configuración:

```
agent1.channels = ch1
```

```
agent1.channels.ch1.type = memory
agent1.channels.ch1.capacity = 100000
agent1.channels.ch1.transactionCapacity = 1000
```

Sumideros(Sinks)

Un sumidero (sink) de Flume extrae los eventos de uno o más canales de Flume y, o bien reenvía eventos a otra fuente de Flume (en un flujo múltiple), o los maneja de forma específica, normalmente para su almacenamiento. Hay una serie de sumideros integrados en Flume, incluyendo HDFS, HBase, Solr y Elasticsearch. El uso de Flume para escribir en Elasticsearch o Solr permite una contar con capacidades de indexación en tiempo real.

```
agent1.sinks = hdfs_sink1

agent1.sinks.hdfs_sink1.type = hdfs
agent1.sinks.hdfs_sink1.hdfs.path = /flume/%y%m%d/%H%M%S
agent1.sinks.hdfs_sink1.hdfs.fileType = DataStream

agent1.sinks.hdfs_sink1.channel = ch1
```


7.2.2. Fiabilidad

La fiabilidad en Flume viene determinada por el tipo de canal que se utilice, si las fuentes de datos tienen la capacidad de publicar o emitir eventos, y si esos eventos se multiplexan a múltiples fuentes con el fin de mitigar los riesgos de fallo irrecuperable de los nodos. Ni los canales de memoria, ni ciertas fuentes, ofrecen fiabilidad y tolerancia a fallos. Si entre los requisitos del sistema estuviera la fiabilidad sería importante sustituir las fuentes menos seguras por otras con buffering, etc. y los canales de memoria por canales con almacenamiento en disco [JS18].

Flume utiliza transacciones separadas para garantizar la entrega desde la fuente al canal y del canal al sumidero. Del mismo modo, se utiliza una transacción para la entrega de los eventos desde el canal al sumidero. Si por alguna razón improbable los eventos no pudieran ser registrados, la transacción se anularía y los eventos permanecerían en el canal para su posterior reenvío. El efecto general es que todos los eventos producidos por la fuente llegarán al sumidero al menos una vez, es decir, es posible que haya duplicados. Esta forma de funcionamiento puede parecer una limitación, pero en la práctica proporciona aceptable compensación de rendimiento. Para asegurar que los eventos llegan exactamente una vez al sumidero se requiere duplicar operaciones lo cual redundante en un perjuicio para el sistema. Con la semántica *at-least-once*, los eventos duplicados pueden ser eliminados más adelante en la cadena de procesamiento. Por lo general, esto adopta forma de un trabajo de deduplicación específico escrito en MapReduce o Hive.

7.2.3. Particiones e Interceptores

Los grandes conjuntos de datos se organizan a menudo en particiones, de modo que el procesamiento puede restringirse a determinadas particiones si sólo se consulta un subconjunto de los datos [Voh16]. Para los datos de eventos de Flume es muy común la partición por tiempo. Se puede ejecutar periódicamente un proceso que transforme las particiones completadas (para eliminar los eventos duplicados, por ejemplo). Es fácil cambiar el ejemplo para almacenar los datos en particiones configurando `hdfs.path` para incluir subdirectorios que utilicen secuencias de escape de formato de tiempo. La partición en la que se escribe un evento de Flume viene determinada por la cabecera `timestamp` del evento.

```
agent1.sinks.sink1.hdfs.path = /tmp/flume/year=%Y/month=%m/day=%d
```

Los eventos no tienen esta cabecera por defecto, pero puede añadirse utilizando un interceptor Flume. Los interceptores son componentes que pueden modificar o eliminar eventos en el flujo; se adjuntan a las fuentes, y se ejecutan en los eventos antes de que éstos hayan sido colocados en un canal. Las siguientes líneas de configuración extra añaden un interceptor de marca de tiempo a `source1`, que añade una cabecera `timestamp` a cada evento producido por la fuente:

```
agent1.sources.source1.interceptors = interceptor1
agent1.sources.source1.interceptors.interceptor1.type = timestamp
```

7.2.4. Configuraciones avanzadas

En la configuración básica, Flume se usa mediante la ejecución de un único agente con una única fuente, un único canal y único sumidero. Flume puede soportar una configuración totalmente distribuida en la cual los agentes se puedan ejecutar múltiples hosts con múltiples conexiones entre la fuente y los destinos finales.

Flujo de Consolidación

Un escenario muy común en la recolección de logs es un gran número de clientes productores de logs que envían datos a unos pocos agentes consumidores que están conectados al subsistema de almacenamiento. Por ejemplo, los logs recogidos de cientos de servidores web se envían a una docena de agentes que escriben en el clúster HDFS. Esto puede lograrse en Flume configurando un número de agentes de primer nivel, todos apuntando a una fuente de un solo agente. Esta fuente en el agente de segundo nivel consolida los eventos recibidos en un único canal que es consumido por un sumidero hasta su destino final. Lo más común es que el formato Avro sea el más utilizado tanto en fuente como en sumidero.

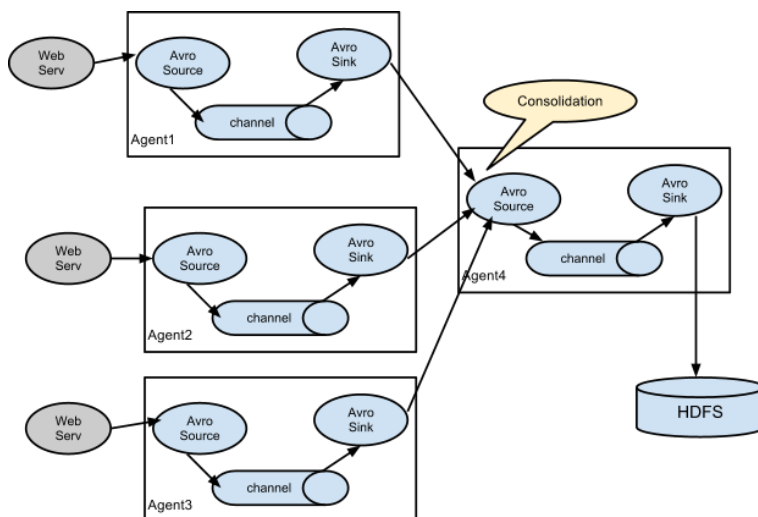


Figura 7.4: Flujo de trabajo de Consolidación de Flume. Fuente: Apache



Avro es un formato de almacenamiento basado en filas para Hadoop que se utiliza como plataforma de serialización. Avro almacena la definición de los datos (esquema) en formato JSON haciendo que sea fácil de leer e interpretar por cualquier programa. Los datos en sí se almacenan en formato binario, lo que los hace compactos y eficientes.

Multiplexado

Flume soporta la multiplexación del flujo de eventos a uno o más destinos. Esto se consigue definiendo un multiplexor de flujo que puede replicar o enrutar selectivamente un evento a uno o más canales.

Por ejemplo se podría contar con una fuente que está conectada con tres canales diferentes. Esta bifurcación puede ser por replicación o por multiplexación. En el caso de un flujo replicado, cada evento se envía a los tres canales. En el caso de la multiplexación, un evento se envía a un subconjunto de canales disponibles cuando el atributo de un evento coincide con un valor preconfigurado. Por ejemplo, si el atributo de un evento llamado *tipoDoc* está configurado como *factura*, entonces debería ir al *channel1* y al *channel3*, si es *albaran* entonces debería ir al *channel2*, en caso contrario al *channel3*. El mapeo se puede establecer en el archivo de configuración del agente.

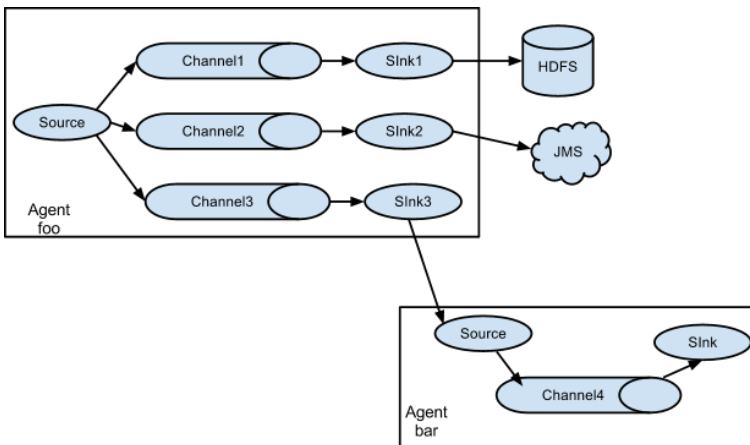


Figura 7.5: Flujo de trabajo Multiplexado. Fuente: Apache

```
agent1.sources.source1.selector.type = multiplexing
agent1.sources.source1.selector.header = State
agent1.sources.source1.selector.mapping.CA = mem-channel-1
agent1.sources.source1.selector.mapping.AZ = file-channel-2
agent1.sources.source1.selector.mapping.NY = mem-channel-1 file-channel-2
agent1.sources.source1.selector.default = mem-channel-1
```

Multiagente

Para que los datos fluyan a través de múltiples agentes el sumidero del primer agente debe ser de tipo Avro y apuntar a un host y puerto concreto tal y como se observa en la siguiente configuración [Hol14].

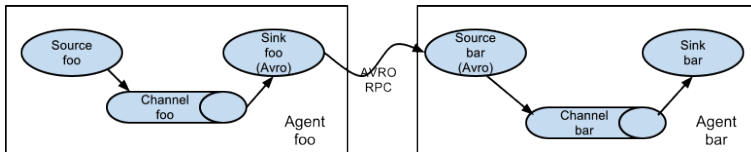


Figura 7.6: Flujo de trabajo Multiagente. Fuente: Apache

```
agent1.sources = source1
agent1.sinks = sink1
agent1.channels = channel1
agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1
agent1.sources.source1.type = spooldir
agent1.sources.source1.spoolDir = /tmp/spooldir
agent1.sinks.sink1.type = avro
agent1.sinks.sink1.hostname = localhost
agent1.sinks.sink1.port = 10000
agent1.channels.channel1.type = file
agent1.channels.channel1.checkpointDir=/tmp/agent1/file-channel/checkpoint
agent1.channels.channel1.dataDirs=/tmp/agent1/file-channel/data
```

La fuente del segundo agente también debe ser del tipo avro y apuntar al mismo host y puerto en el que el sumidero vuelva sus resultados.

```
agent2.sources = source2
agent2.sinks = sink2
agent2.channels = channel2
agent2.sources.source2.channels = channel2
agent2.sinks.sink2.channel = channel2
agent2.sources.source2.type = avro
agent2.sources.source2.bind = localhost
agent2.sources.source2.port = 10000
```

...

Dado que hay dos canales de tipo archivo que se ejecutan en la misma máquina, están configurados para apuntar a diferentes directorios de datos y puntos de control, de esta manera, no intentan escribir sus archivos uno encima del otro.

Cada agente se ejecuta de forma independiente, utilizando el mismo parámetro *-conf-file* pero diferentes nombres de agente.

Bibliografía

- [Flu] Apache Flume. User guide. <http://flume.apache.org/flumeuserguide.html>.
- [Hof13] Steve Hoffman. *Apache Flume: distributed log collection for Hadoop*. Packt Publishing Ltd, 2013.
- [Hol14] Alex Holmes. *Hadoop in practice*. Simon and Schuster, 2014.
- [JS18] Sooyong Jung and Yongtae Shin. Study of the big data collection scheme based apache flume for log collection. *International Journal of Computer Theory and Engineering*, 10(3):97–100, 2018.
- [ORS⁺08] Christopher Olston, Benjamin Reed, Utkarsh Srivastava, Ravi Kumar, and Andrew Tomkins. Pig latin: a not-so-foreign language for data processing. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 1099–1110, 2008.
- [Vad16] Balaswamy Vaddeman. *Beginning Apache Pig*. Springer, 2016.
- [Voh16] Deepak Vohra. Apache flume. In *Practical Hadoop ecosystem*, pages 287–300. Springer, 2016.